

ScoutForce - Dokumentacja

Grupa: WIs I.6 - 16c, 2w

Betlej Filip s30331

Maj 2026

Spis Treści

1	Wprowadzenie	3
1.1	Dziedzina problemowa	3
1.2	Cel	3
1.3	Zakres odpowiedzialności systemu	3
1.4	Użytkownicy systemu	3
1.5	Konwencje nazewnictwa	3
1.5.1	Język i zakres stosowania	3
1.5.2	Przypadki użycia (Use Cases)	4
1.5.3	Klasy i obiekty domenowe	4
1.5.4	Atrybuty i pola danych	4
1.5.5	Aktorzy i role	4
1.5.6	Terminy domenowe w języku polskim	4
2	Wymagania użytkownika	5
2.1	Osoby i pracownicy	5
2.2	Kluby	5
2.3	Zawodnicy	5
2.4	Mecze i statystyki	6
2.5	Raporty skautingowe	6
2.6	Delegacje	7
2.7	Funkcjonalności	7
3	Diagramy przypadków użycia	8
4	Wymagania нефunkcjonalne	9
4.1	Wydańnościowe	9
4.2	Niezawodnościowe	9
4.3	Bezpieczeństwa	9
4.4	Użyteczności	9
4.5	Przenośności	9
4.6	Skalowalności i architektury	10
5	Analityczny diagram klas	11
6	Projektowy (implementacyjny) diagram klas	12
7	Scenariusze przypadków użycia	13
7.1	Create Scouting Report	13
7.1.1	Informacje ogólne	13
7.1.2	Powiązane przypadki użycia	13
7.1.3	Scenariusz główny	14
7.1.4	Scenariusze alternatywne	15
7.1.5	Scenariusze wyjątków	15
7.2	Add Detailed Ratings	17
7.2.1	Informacje ogólne	17
7.2.2	Powiązane przypadki użycia	17
7.2.3	Scenariusz główny	18
7.2.4	Scenariusze wyjątków	18
8	Diagramy aktywności	20
8.1	Create Scouting Report	20
8.2	Add Detailed Ratings	21

9	Diagram stanu dla klasy Player	22
10	Projekt interfejsu użytkownika (GUI)	23
10.1	Ekran główny aplikacji po kliknięciu w zawodnika	23
10.2	Create Scouting Report	23
10.3	Stany walidacji przy zatwierdzaniu raportu	25
10.4	Potwierdzenie anulowania raportu (E6)	25
10.5	Potwierdzenie zapisu raportu - sukces	26
10.6	Wyjątek E1 - brak meczów obserwowanych przez skauta (widok Create Report) ..	26
10.7	Widok meczów zawodnika - brak meczów (scenariusz alternatywny A1 + E1)	27
11	Omówienie decyzji projektowych i skutków analizy dynamicznej	28
11.1	Wykorzystane technologie	28
11.2	Klasa Player - dziedziczenie dynamiczne i ograniczenia atrybutów	28
11.2.1	Dynamiczne dziedziczenie zawodnika (<<dynamic>>)	28
11.3	Ograniczenia atrybutów typu wyliczeniowego - enumy	29
11.3.1	Sposób przechowywania enumów w bazie	30
11.4	Mapowanie asocjacji i agregacji w Hibernate	30
11.4.1	Ekstensja klas a persystencja (decyzja implementacyjna)	30
11.5	Klasa Scouting Report - kompozycja, klasa pośrednicząca i atrybuty pochodne ...	31
11.5.1	Klasa pośrednicząca Match Stats	31
11.6	Kompozycja Scouting Report - Detailed Rating	31
11.6.1	Walidacja sumy wag - <code>validateDetailedRatings()</code>	32
11.7	Atrybuty pochodne - metody <code>get{AtrybutPochodny}()</code>	32
11.8	Ograniczenia unikalności (unique)	32
11.9	Implementacja atrybutów klasowych, dostępu do danych i metod obiektowych	32

1 Wprowadzenie

1.1 Dziedzina problemowa

System ScoutForce znajduje zastosowanie w dziedzinie profesjonalnego skautingu koszykarskiego w lidze NBA. Wspiera on pracę działów skautingu klubów NBA, odpowiedzialnych za identyfikację, obserwację i ewaluację potencjalnych talentów draftowych - zarówno z uczelni amerykańskich (NCAA), lig zagranicznych, jak i niższych rozgrywek profesjonalnych (np. G-League).

1.2 Cel

Celem budowy systemu jest usprawnienie i usystematyzowanie pełnego cyklu pracy skautingowej: od planowania wyjazdów obserwacyjnych (delegacji), przez rejestrowanie statystyk meczowych i tworzenie raportów skautingowych, po generowanie rankingów draftowych. System ma wyeliminować rozproszone arkusze kalkulacyjne i niespójne notatki, zapewniając jednolite środowisko pracy dla skautów i dyrektorów sportowych.

1.3 Zakres odpowiedzialności systemu

System odpowiada za: zarządzanie danymi zawodników i ich statusami w procesie skautingowym, rejestrowanie meczów i szczegółowych statystyk występów, tworzenie raportów skautingowych z wielowymiarowymi ocenami, planowanie i rozliczanie delegacji skautingowych, prowadzenie analiz strzeleckich per sezon i strefa boiska oraz generowanie rankingów draftowego (Big Board) na podstawie zgromadzonych ocen.

1.4 Użytkownicy systemu

Potencjalni użytkownicy systemu to:

- **Dyrektor sportowy klubu (Director** w notacji UML) - osoba zarządzająca działem skautingu, odpowiedzialna za planowanie delegacji i podejmowanie decyzji draftowych.
- **Skaut (Scout** w notacji UML) - pracownik terenowy obserwujący zawodników na meczach, tworzący raporty skautingowe i wprowadzający statystyki meczowe.

1.5 Konwencje nazewnictwa

W całej dokumentacji przyjęto rozdzielenie warstwy opisu biznesowego (język polski) od warstwy modelowania i implementacji (język angielski). Poniższe reguły obowiązują we wszystkich rozdziałach.

1.5.1 Język i zakres stosowania

- **Opis narracyjny** (wymagania użytkownika, wprowadzenie, komentarze i przepływ scenariuszy, komunikaty w scenariuszu) - **polski**.
- **Elementy modelu UML** (aktorzy, przypadki użycia, klasy, atrybuty, stany, relacje <<include>> / <<extend>> oraz komunikaty w GUI) - **angielski**.

- W scenariuszach przypadków użycia aktor wykonujący kroki oznaczany jest jako **Scout** lub **Director** (zgodnie z diagramem przypadków użycia), a nie „Skaut„ / „Dyrektor”.

1.5.2 Przypadki użycia (Use Cases)

- Nazwy w notacji **Title Case**, po angielsku, zgodne z diagramem przypadków użycia, np. Create Scouting Report, View All Players, Add Detailed Ratings, View Player Matches.
- W tekście scenariusza odwołania do przypadków użycia zapisywane są w backtickach, np. Create Scouting Report.
- Relacje UML: <<include>>, <<extend>> - nazwa docelowa po angielsku.

1.5.3 Klasy i obiekty domenowe

- Nazwy klas w diagramach: **PascalCase** z odstępem między słowami, np. Player, Scouting Report, Detailed Rating, Match Stats, Delegation, Club.

1.5.4 Atrybuty i pola danych

- Wartości wyliczeniowe (enum) w modelu UML i implementacji (Java): **UPPERCASE** ze znakiem podkreślenia, zgodnie z konwencją stałych enumów Javy, np. STRONG_BUY, BUY, NEUTRAL, PASS oraz statusy zawodnika: NEW, OBSERVED, MEDICAL_VERIFICATION, INVITED_TO_WORKOUT, INVITED_TO_BIG_BOARD, DELISTED.
- W **wymaganiach funkcjonalnych** (rozdz. 2) wartości enum mogą być podane w pisowni naturalnej (np. "strong buy", "new") - bez konieczności stosowania UPPERCASE.

1.5.5 Aktorzy i role

Opis (PL, narracja)	Aktor / rola (UML, EN)
Skaut	Scout
Dyrektor sportowy klubu / dyrektor klubu	Director

1.5.6 Terminy domenowe w języku polskim

Poniższe pojęcia w tekście polskim **nie są tłumaczone** na angielski w narracji, lecz mają przypisane nazwy techniczne w modelu:

Termin (PL)	Odpowiednik w modelu UML
Raport skautingowy	Scouting Report
Ocena szczegółowa	Detailed Rating
Statystyki meczowe (występ zawodnika)	Match Stats
Zawodnik	Player
Mecz	Match
Delegacja	Delegation
Klub	Club
Ranking draftowy (Big Board)	lista zawodników o statusie INVITED_TO_BIG_BOARD

2 Wymagania użytkownika

Dyrektor sportowy klubu NBA postanowił zamówić system „ScoutForce” wspomagający pracę działu skautingu oraz zarządzanie ewaluacją potencjalnych talentów draftowych. System ma usprawnić pełen cykl pracy skautingowej: od planowania delegacji, przez obserwację zawodników na meczach, po tworzenie raportów skautingowych z indywidualnymi rekomendacjami draftowymi.

2.1 Osoby i pracownicy

1. W systemie należy przechowywać dane osobowe i dane kontaktowe wszystkich osób związanych z procesem skautingu. Zakłada się, że dane osobowe zawierają datę urodzenia. Dla każdej osoby pamiętamy dodatkowo unikatowy adres email.
2. Wszystkie osoby zarejestrowane w systemie podzielić można, ze względu na rolę w organizacji, na pracowników klubu oraz zawodników. **Podział jest kompletny i rozłączny.**
3. Pracownicy klubu dzielą się dalej m.in. na dyrektorów i skautów. Każdy pracownik jest dokładnie jednym z tych dwóch typów, w przyszłości mogą zostać dodane kolejne typy pracownika. Dla każdego pracownika oprócz danych osobowych pamiętamy: datę zatrudnienia oraz dzienną stawkę wynagrodzenia.
4. Dla skauta pamiętamy dodatkowo: unikatowy numer licencji skautingowej, specjalizację oraz region obserwacji.
5. **Tworzenie nowych delegacji jest realizowane wyłącznie przez dyrektora klubu** - to dyrektor decyduje, którego skauta wysłać i na jaki okres.

2.2 Kluby

6. W systemie pamiętane są informacje o klubach koszykarskich (zawodowych zespołach NBA, klubach uczelnianych NCAA oraz klubach z lig zagranicznych). Dla każdego klubu pamiętamy: nazwę, miasto oraz ligę. Dla niektórych klubów (zespołów NBA) pamiętamy dodatkowo konferencję i dywizję - w pozostałych ligach informacje te nie są istotne.
7. Każdy pracownik klubu jest zatrudniony w dokładnie jednym klubie. Każdy zawodnik aktualnie reprezentowany w systemie gra w dokładnie jednym klubie.

2.3 Zawodnicy

8. Dla każdego zawodnika oprócz danych osobowych pamiętamy: pozycję na boisku, status w procesie skautingowym, wagę, wzrost oraz rozpiętość ramion. **Dla każdego zawodnika można obliczyć średnią ocenę** wynikającą ze wszystkich raportów skautingowych dotychczas wystawionych dla tego zawodnika.
9. Zawodnicy dzielą się, ze względu na doświadczenie, na zawodników uniwersyteckich (z amerykańskich uczelni NCAA) oraz zawodników profesjonalnych (np. G-League, Euroleague). **Podział jest kompletny i rozłączny, ale dynamiczny** - zawodnik kończąc college przestaje być zawodnikiem uniwersyteckim i z reguły staje się zawodnikiem profesjonalnym

lub międzynarodowym (przykładowo: zawodnik z UCLA podpisujący kontrakt z drużyną EuroLeague, może to być również w drugą stronę).

10. Dla zawodnika uniwersyteckiego pamiętamy dodatkowo uczelnię oraz rocznik (freshman, sophomore, junior, senior). Dla zawodnika profesjonalnego pamiętamy kraj pochodzenia.
11. Każdy zawodnik posiada w systemie status z procesu skautingowego: „new”, „observed”, „medical verification”, „invited to workout”, „invited to Big Board”, „delisted”. Pomiedzy statusami zdefiniowano dozwolone przejścia (np. przejście „new → observed” wymaga utworzenia co najmniej jednego raportu skautingowego dla danego zawodnika).

2.4 Mecze i statystyki

12. Skauci obserwują zawodników podczas meczów. Dla każdego meczu pamiętamy datę, miejsce, wynik gospodarza oraz wynik gościa. **W każdym meczu biorą udział dokładnie dwie drużyny** - jedna w roli gospodarza, druga w roli gościa - wybierane spośród klubów istniejących w systemie.
13. Dla każdego występu zawodnika w meczu pamiętamy zestaw statystyk koszykarskich: liczbę rozegranych minut, punkty, zbiórki, asysty, przechwyty, bloki, straty, faule oraz dla każdego rodzaju rzutu (z gry, za 3 punkty, wolnych) liczbę rzutów oddanych i trafionych. Statystyki te przechowujemy w sposób umożliwiający jednoznaczne powiązanie ich z konkretnym zawodnikiem oraz konkretnym meczem (występ konkretnego zawodnika w konkretnym meczu). **Dla każdego rodzaju rzutu można obliczyć procent skuteczności** na podstawie liczby rzutów oddanych i trafionych.
14. Dodatkowo dla każdego zawodnika prowadzona jest analiza strzelecka per sezon i per dystans. Analiza składa się z liczby rzutów oddanych i trafionych w danym sezonie i strefie. **Dla analizy również można obliczyć procent skuteczności** w zadanym sezonie i strefie.

2.5 Raporty skautingowe

15. Po obejrzeniu występów zawodnika w meczach skaut tworzy na ich podstawie raport skautingowy. **Raport jest powiązany z dokładnie jednym skautem, jednym zawodnikiem** Dla raportu pamiętamy datę utworzenia, notatkę tekstową oraz rekomendację draftową. **Dla każdego raportu można obliczyć ocenę końcową** na podstawie ocen szczegółowych zawartych w raporcie.
16. Rekomendacja draftowa przyjmuje jedną z czterech wartości: „strong buy” (zdecydowanie pozyskać), „buy” (pozyskać), „neutral” (neutralna) lub „pass” (pomiąć).
17. Raport skautingowy zawiera od jednej do wielu ocen szczegółowych. **Ocena szczegółowa istnieje wyłącznie w kontekście raportu, do którego została dołączona - usunięcie raportu skutkuje automatycznym usunięciem wszystkich jego ocen szczegółowych.** Każda ocena szczegółowa posiada: typ oceny (definiowany swobodnie przez skauta - np. „ofensywa”, „obrona”, „atletyzm”, „IQ koszykarskie”, „charakter”), wartość liczbową w skali 1-10, komentarz tekstowy oraz wagę z przedziału 0.0-1.0.
18. **Wagi wszystkich ocen szczegółowych w obrębie jednego raportu muszą sumować się do 1.0.** Ocena końcowa raportu jest liczona jako średnia ważona ocen szczegółowych.

19. Każdy skaut ma swobodę w doborze zestawu kategorii ocen szczegółowych - jeden skaut może oceniać 3 wymiary, inny 5, w zależności od własnej metodologii skautingowej.

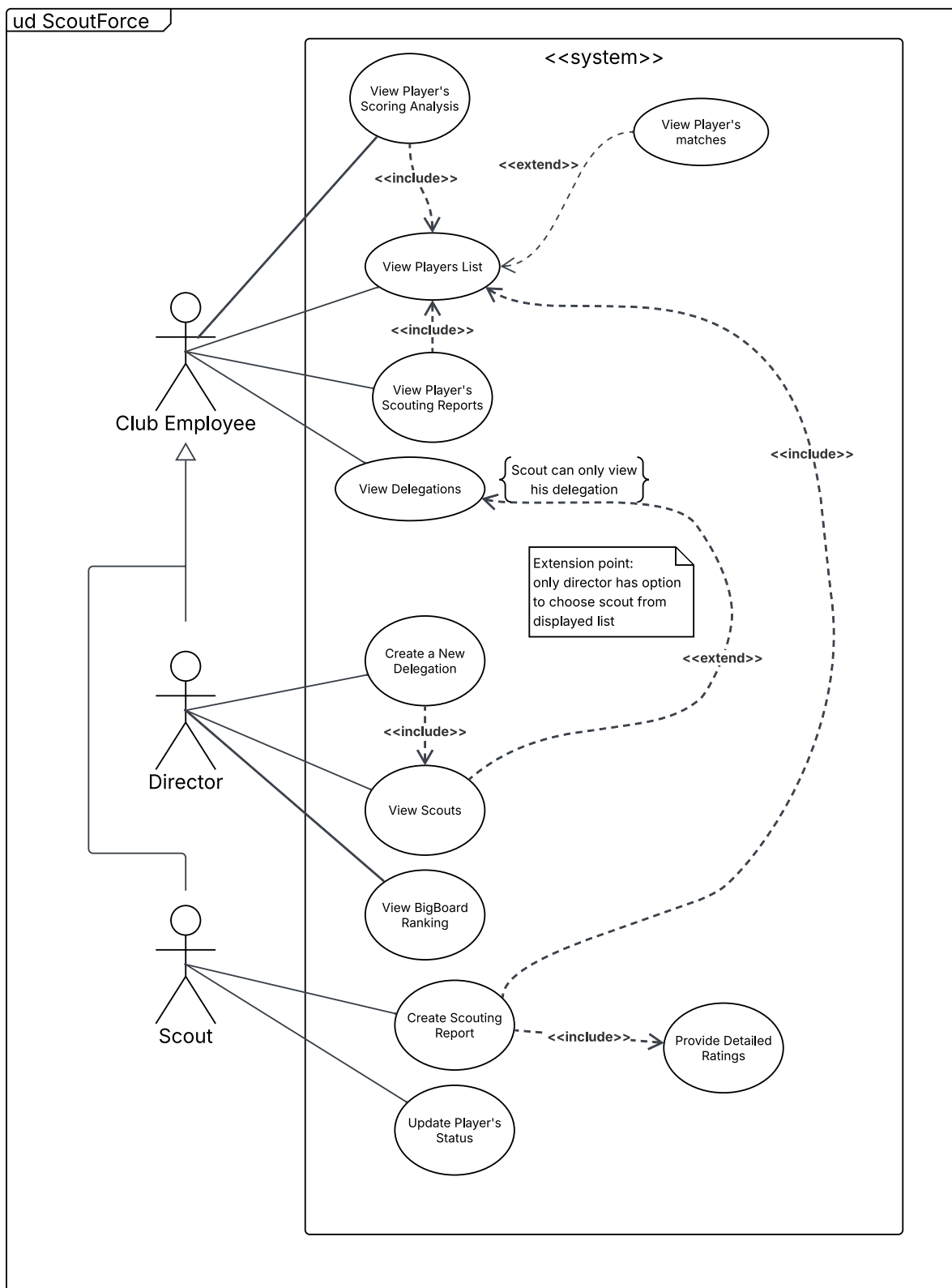
2.6 Delegacje

20. Skauci wyjeżdżają na delegacje, w trakcie których obserwują wybrane mecze. Dla każdej delegacji pamiętamy: nazwę, datę rozpoczęcia, datę zakończenia oraz status. **Każda delegacja jest organizowana dla dokładnie jednego skauta i utworzona przez dokładnie jednego dyrektora klubu.**
21. **Dla każdej delegacji można obliczyć jej koszt.** Koszt liczony jest jako: liczba dni delegacji pomnożona przez dzienną stawkę skauta, plus stałe koszty utrzymania pomnożone przez liczbę dni, plus stały koszt przelotu.
22. **Stały koszt utrzymania jest jednakowy dla wszystkich delegacji** w skali klubu; aktualnie wynosi 250\$ za dzień. Wartość ta może w przyszłości ulec zmianie.
23. **Stały koszt przelotu jest jednakowy dla wszystkich delegacji** w skali klubu; aktualnie wynosi 1500\$. Wartość ta również może w przyszłości ulec zmianie.
24. W ramach jednej delegacji skaut może obserwować wiele meczów. **Ten sam mecz może być również obserwowany przez wielu skautów w ramach różnych delegacji.**

2.7 Funkcjonalności

25. System powinien wspierać użytkowników w realizowaniu m.in. następujących funkcjonalności:
- 25.1. **Utworzenie nowej delegacji** dla wybranego skauta z określonym zakresem dat (dyrektor klubu).
- 25.2. **Wyświetlenie listy skautów** (dyrektor klubu).
- 25.3. **Stworzenie raportu skautingowego** dla zawodnika (skaut) wyświetlonego z listy, wraz z:
- dołączeniem od jednej do wielu ocen szczegółowych (z walidacją sumy wag = 1.0),
 - automatycznym wyliczeniem oceny końcowej,
- 25.4. **Wyświetlenie historii delegacji** danego skauta wraz z meczami obserwowanymi w trakcie każdej delegacji oraz raportami utworzonymi w ich ramach (skaut, dyrektor klubu).
- 25.5. **Wyświetlenie listy wszystkich zawodników** wraz z aktualnymi statusami oraz średnimi ocenami (skaut, dyrektor klubu).
- 25.6. **Wyświetlenie analizy strzeleckiej** zawodnika dla wybranego sezonu i strefy boiska (skaut, dyrektor klubu).
- 25.7. **Wyświetlenie raportów skautingowych** dla wybranego zawodnika wraz ze szczegółowymi ocenami i komentarzami (skaut, dyrektor klubu).
- 25.8. **Wyświetlenie listy zawodników o statusie „Zaproszony na Big Board”** posortowanej według średniej oceny - stanowiącej de facto ranking draftowy klubu (dyrektor klubu).
- 25.9 **Aktualizacja statusu zawodnika** w celu ułatwienia zarządzania Big Board (skaut).

3 Diagramy przypadków użycia



Rysunek 1: Diagram przypadków użycia systemu ScoutForce

4 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne dla systemu ScoutForce sformułowano w sposób mierzalny i realistycznie weryfikowalny.

4.1 Wydajnościowe

1. **Czas odpowiedzi dla operacji odczytu:** dla typowych zapytań (wyświetlenie listy zawodników, listy raportów skautingowych, listy delegacji) system zwraca odpowiedź w czasie **poniżej 2 sekund** przy zbiorze do 1000 rekordów w bazie danych.
2. **Czas zapisu raportu skautingowego:** zapis nowego raportu skautingowego wraz z ocenami szczegółowymi i aktualizacją statusu zawodnika trwa **poniżej 1 sekundy**.
3. **Obsługa równoległych użytkowników:** system poprawnie obsługuje co najmniej **10 jednoczesnych sesji użytkowników** bez utraty spójności danych.
4. **Wydajność obliczeń pochodnych:** wyliczenie atrybutów pochodnych (/averageRating zawodnika, /finalRating raportu, /cost delegacji) odbywa się w czasie **poniżej 200 ms** dla zawodnika z maksymalnie 100 raportami.

4.2 Niezawodnościowe

5. **Transakcyjność zapisów:** wszystkie operacje zmieniające stan systemu (utworzenie raportu, zmiana statusu zawodnika, utworzenie delegacji) muszą być atomowe - przerwanie operacji nie pozostawia bazy w niespójnym stanie.
6. **Walidacja danych po stronie serwera oraz klienta:** każda operacja zapisu jest poprzedzona walidacją (np. suma wag ocen szczegółowych równa 1.0, unikatowość pól email i license_number, poprawność dat w delegacji). Nieprawidłowe dane są odrzucane z czytelnym komunikatem błędu.

4.3 Bezpieczeństwa

7. **Autoryzacja w oparciu o role:** system rozróżnia co najmniej dwie role - Skaut oraz Dyrektor klubu. Tworzenie delegacji jest dostępne wyłącznie dla roli Dyrektor klubu.
8. **Ochrona danych w transporcie:** komunikacja klient - serwer w środowisku produkcyjnym odbywa się przez **HTTPS**.

4.4 Użyteczności

9. **Nawigacja:** każda funkcjonalność wymieniona w wymaganiach użytkownika jest dostępna w maksymalnie 3 kliknięciach od ekranu głównego.

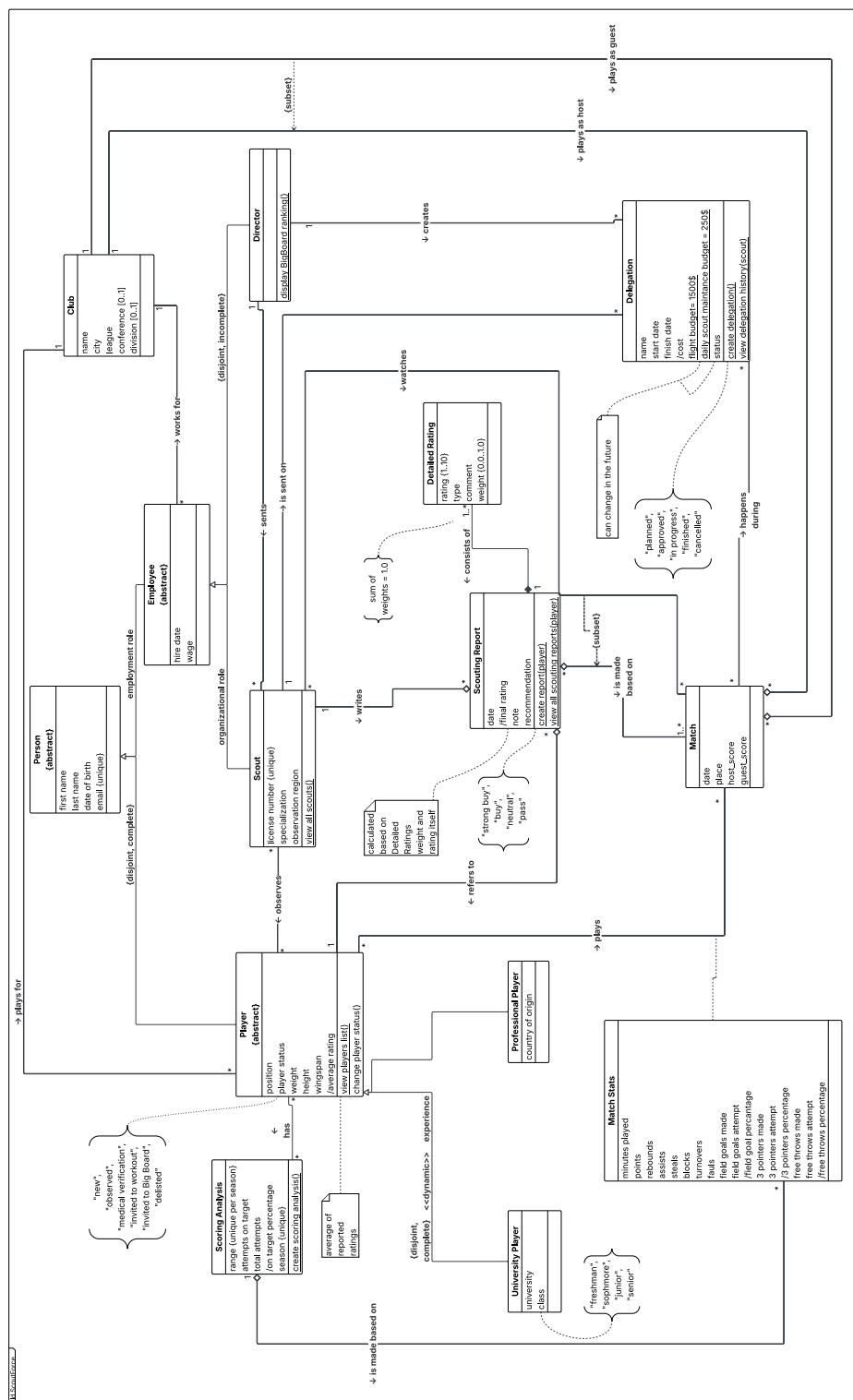
4.5 Przenośności

10. **Środowisko docelowe:** system działa na systemach operacyjnych **Windows 10+**, **Linux (Ubuntu 22.04+)**, **macOS 13+**, **Android 14+**, **iOS 18+**.

4.6 Skalowalności i architektury

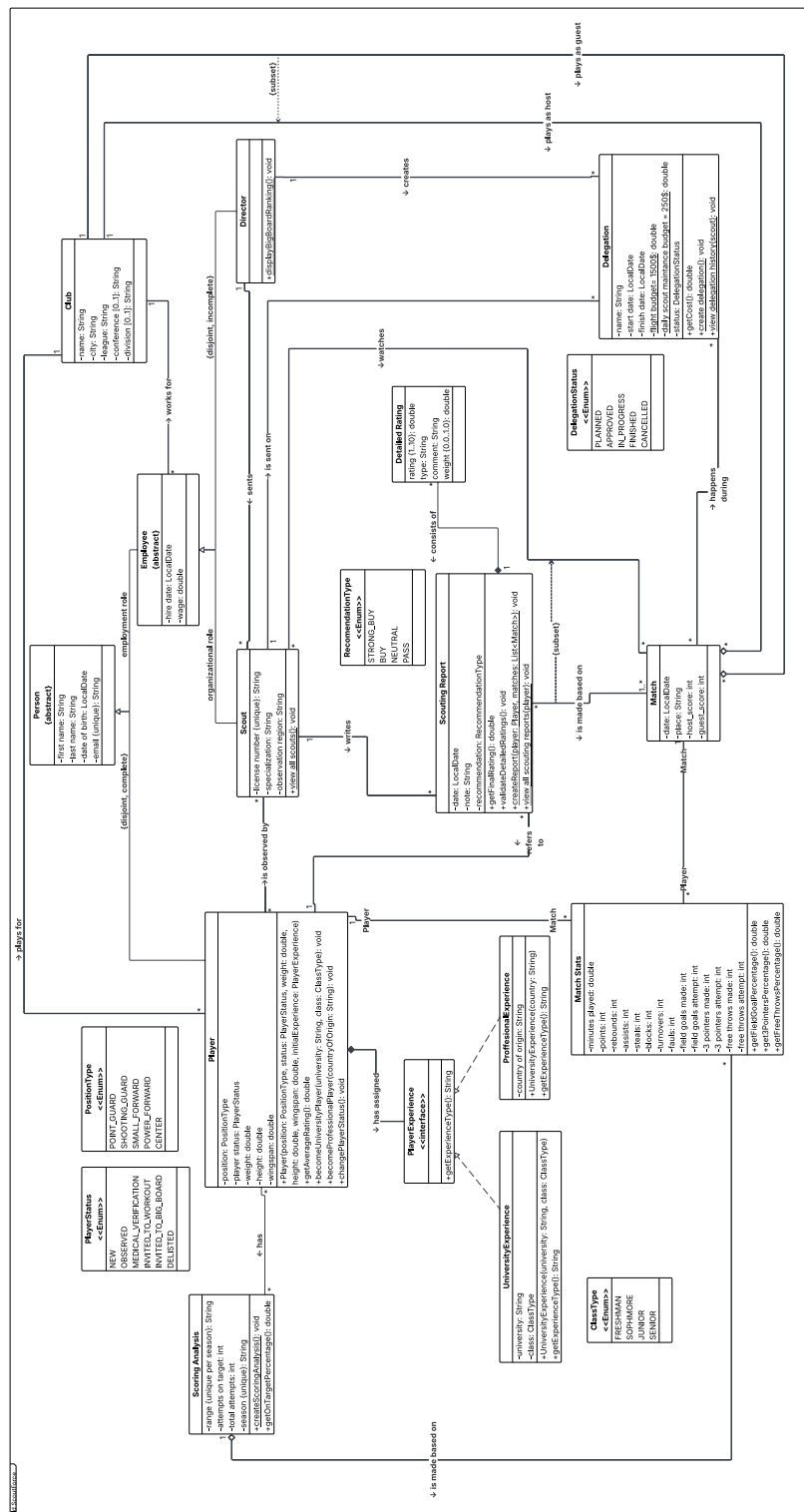
11. **Niezależność warstw:** warstwa prezentacji komunikuje się z backendem wyłącznie przez zdefiniowane API REST - bez bezpośredniego dostępu do bazy.

5 Analityczny diagram klas



Rysunek 2: Analityczny diagram klas systemu ScoutForce

6 Projektowy (implementacyjny) diagram klas



Rysunek 3: Projektowy diagram klas systemu ScoutForce

7 Scenariusze przypadków użycia

7.1 Create Scouting Report

7.1.1 Informacje ogólne

Nazwa	Create Scouting Report
Aktor główny	Scout
Aktorzy drugorzędni	brak
Cel	Utworzenie i zatwierdzenie raportu skautingowego (Scouting Report) dla wybranego zawodnika (Player) na podstawie obserwacji jego występów.
Wyzwalacz	Scout decyduje się utworzyć raport skautingowy zawodnika po obserwacji jego występu w co najmniej jednym meczu.
Warunki początkowe	- Scout obserwował dotychczas co najmniej jednego zawodnika (istnieje co najmniej jedna instancja Match Stats dla zawodnika w meczu, który Scout obserwował). - Scout obejrzał dotychczas co najmniej jeden mecz.
Warunki końcowe - sukces	- W systemie istnieje nowy obiekt Scouting Report powiązany z zalogowanym Scoutem oraz wybranym zawodnikiem (Player). - Raport zawiera co najmniej jedną ocenę szczegółową (Detailed Rating); suma wag wszystkich ocen w raporcie jest równa dokładnie 1.0. - Atrybut pochodny /finalRating raportu jest możliwy do wyliczenia na podstawie ocen szczegółowych. - Atrybut recommendation raportu przyjmuje jedną z wartości: STRONG_BUY, BUY, NEUTRAL, PASS.
Warunki końcowe - porażka	Stan systemu nie zmienia się - żaden Scouting Report ani żadna instancja Detailed Rating nie zostają zapisane, atrybuty pochodne zawodnika pozostają bez zmian.

7.1.2 Powiązane przypadki użycia

- <<include>> View All Players - wyświetlenie listy zawodników jako punkt wejścia do wyboru obiektu raportu.
- <<include>> Add Detailed Ratings - obowiązkowa część procesu (raport musi mieć 1..* ocen szczegółowych).
- <<extend>> View Player Matches - opcjonalne rozszerzenie umożliwiające zawężenie raportu do podzbioru meczów wybranego ręcznie przez Scouta.

7.1.3 Scenariusz główny

Wariant domyślny - raport tworzony bezpośrednio z poziomu panelu zawodnika, **bez przeglądania listy meczów**. Zakres raportu obejmuje **wszystkie** mecze, w których wystąpił wybrany zawodnik i które jednocześnie obserwował Scout.

1. Scout wybiera z menu opcję przeglądania zawodników. **System wywołuje** <<include>> View All Players.
2. System wyświetla listę zawodników. Dla każdej pozycji prezentowane są: pola identyfikujące (`first_name`, `last_name`, `position`, `club`, `player_status`) oraz atrybut pochodny: `/averageRating`.
3. Scout wybiera zawodnika z listy.
4. System wyświetla panel wybranego zawodnika z dwiema akcjami dostępnymi natychmiast:
 - „Create Scouting Report„ - utworzenie raportu obejmującego **wszystkie** mecze zawodnika obserwowane przez Scouta (przeływ domyślny, kontynuuje krok 5);
 - „View Player Matches„ - opcjonalne przejście do listy meczów w celu zawężenia raportu do wybranych meczów (przeływ alternatywny A1).
5. Scout wybiera akcję „Create Scouting Report„.
6. System ustala zakres meczów raportu jako pełny zbiór meczów spełniających dwa warunki: zawodnik wystąpił w meczu (istnieje obiekt `Match Stats` dla pary (*Player*, *Match*)) oraz Scout obserwował ten mecz. *Jeżeli zbiór jest pusty - wyjątek E1*.
7. System wyświetla formularz nowego raportu z polami: notatka tekstowa (`note`), rekomendacja draftowa (`recommendation`), dynamiczna sekcja ocen szczegółowych oraz panel statystyk meczów wchodzących w zakres raportu (uśrednione `minutes_played`, `points`, `rebounds`, `assists`, `steals`, `blocks` z odpowiednich `Match Stats`).
8. Scout dodaje pierwszą ocenę szczegółową. **System wywołuje** <<include>> Add Detailed Ratings - przyjmuje od Scouta: `type` (kategoria swobodna, np. „*offense*„, „*defense*“, „*athleticism*„, „*basketball IQ*“, „*character*„), `rating`: [1, 10], `comment` oraz `weight`: [0.0, 1.0]. *Jeżeli wartość pola jest poza zakresem - wyjątek E4*.
9. Scout powtarza krok 8 dla kolejnych kategorii oceny (1..* ocen szczegółowych - co najmniej jedna wymagana).
10. Scout wypełnia pole `note` opisem słownym swojej obserwacji.
11. Scout wybiera z listy rozwijanej wartość pola `recommendation`: `STRONG_BUY`, `BUY`, `NEUTRAL` lub `PASS`.
12. Scout zatwierdza formularz przyciskiem „Zatwierdź raport„.
13. System weryfikuje, czy **suma** atrybutów `weight` wszystkich pozycji `Detailed Rating` w raporcie jest równa dokładnie 1.0. *Jeżeli suma jest inna - wyjątek E3*.
14. System zapisuje nowy obiekt `Scouting Report` wraz z powiązanymi obiektami `Detailed Rating`.
15. System wyświetla potwierdzenie zapisu raportu wraz z wyliczonym `/finalRating`.
16. Przypadek użycia kończy się sukcesem.

7.1.4 Scenariusze alternatywne

7.1.4.1 A1. Scout zawęźa raport do wybranych meczów (<<extend>> View Player Matches)

Punkt rozszerzenia: Wybór zakresu meczów (krok 4 scenariusza głównego). Rozszerzenie jest **opcjonalne** - Scout może je pominąć i utworzyć raport bezpośrednio (krok 5 scenariusza głównego).

1. Scout w kroku 4 wybiera akcję „View Player Matches”, zamiast „Create Scouting Report”. **System wywołuje <<extend>> View Player Matches.**
2. System wyświetla listę meczów wybranego zawodnika **ograniczoną** do meczów obserwowanych przez Scouta. Dla każdego meczu prezentowane są: date, place, host_score : guest_score, nazwa klubu gospodarza i gościa oraz statystyki indywidualne zawodnika z Match Stats (minutes_played, points, rebounds, assists, steals, blocks). *Jeżeli lista jest pusta - wyjątek E1.*
3. Scout zaznacza wybrane mecze (co najmniej jeden mecz wymagany).
4. Scout wybiera akcję „Create Scouting Report from selected matches”.
5. System ustala zakres meczów raportu jako zbiór dokładnie tych meczów, które Scout zaznaczył (zamiast pełnego zbioru obserwowanych meczów zawodnika).
6. Scenariusz wraca do kroku 7 scenariusza głównego.

7.1.5 Scenariusze wyjątków

7.1.5.1 E1. Wybrany zawodnik nie ma żadnego meczu obserwowanego przez Scouta

W kroku 6 scenariusza głównego lub w kroku A1.2 scenariusza alternatywnego, jeżeli ustalony zbiór meczów wchodzących w zakres raportu jest pusty:

1. System wyświetla komunikat: „Wybrany zawodnik nie ma żadnego meczu, który obserwować. Aby utworzyć raport, najpierw zaobserwuj co najmniej jeden mecz tego zawodnika.”
2. System wraca do panelu zawodnika (krok 4 scenariusza głównego).
3. Scout może wybrać innego zawodnika lub zakończyć pracę.
4. Przypadek użycia kończy się niepowodzeniem (warunki końcowe - porażka).

7.1.5.2 E2. Scout nie dodał żadnej oceny szczegółowej

W kroku 13 scenariusza głównego, jeżeli kolekcja Detailed Rating raportu jest pusta:

1. System wyświetla komunikat walidacji: „Raport musi zawierać co najmniej jedną ocenę szczegółową. Dodaj przynajmniej jedną ocenę przed zatwierdzeniem raportu.”
2. System pozostawia formularz w trybie edycji bez zapisu danych.
3. Scenariusz wraca do kroku 8 scenariusza głównego.

7.1.5.3 E3. Suma wag ocen szczegółowych jest różna od 1.0

W kroku 13 scenariusza głównego, podczas walidacji:

1. System wyświetla komunikat: „Suma wag ocen szczegółowych musi wynosić dokładnie 1.0. Aktualna suma: <obecna wartość>.”

2. System pozostawia formularz w trybie edycji.
3. Scenariusz wraca do kroku 8 scenariusza głównego (w celu korekty wag).

7.1.5.4 E4. Wartość oceny lub wagi poza dopuszczalnym zakresem, brak komentarza

W kroku 8 scenariusza głównego, podczas wprowadzania pojedynczej oceny szczegółowej, jeżeli niespełnione warunki: `rating: [1, 10]`, `weight: [0.0, 1.0]` lub brak `comment`:

1. System wyświetla komunikat walidacji pola: „Ocena musi mieścić się w zakresie 1-10. Waga musi mieścić się w zakresie 0.0-1.0.” lub „Ocena musi mieć komentarz.”
2. System uniemożliwia dodanie nieprawidłowej oceny do raportu.
3. Scout koryguje wartość - scenariusz pozostaje w kroku 8.

7.1.5.5 E5. Brak wybranej rekomendacji draftowej, brak notki

W kroku 11 scenariusza głównego, jeżeli pole `recommendation` nie zostało wypełnione (`recommendation = NULL`) lub pole `note` jest puste:

1. System wyświetla komunikat walidacji: „Rekomendacja draftowa jest wymagana. Wybierz jedną z opcji: `STRONG_BUY`, `BUY`, `NEUTRAL`, `PASS`.” lub „Notka jest wymagana. Wypełnij notkę.”
2. System pozostawia formularz w trybie edycji.
3. Scenariusz wraca do kroku 11 scenariusza głównego.

7.1.5.6 E6. Scout anuluje operację

W dowolnym kroku po kroku 5 scenariusza głównego:

1. Scout wybiera akcję „Anuluj”.
2. System wyświetla potwierdzenie: „Czy na pewno chcesz anulować raport? Wprowadzone dane zostaną utracone.”
3. Jeżeli Scout potwierdza - system odrzuca wszystkie wprowadzone dane i wraca do panelu zawodnika (krok 4).
4. Jeżeli Scout wycofuje anulowanie - scenariusz wraca do ostatnio aktywnego kroku.
5. Przypadek użycia kończy się niepowodzeniem (warunki końcowe - porażka).

7.2 Add Detailed Ratings

7.2.1 Informacje ogólne

Nazwa	Add Detailed Ratings
Aktor główny	Scout
Aktorzy drugorzędni	brak
Cel	Wprowadzenie w formularzu tworzonego raportu skautingowego (Scouting Report) pojedynczej pozycji oceny szczegółowej (Detailed Rating) w wybranej kategorii - wraz z komentarzem oraz wagą określającą udział tej oceny w /finalRating - tak aby została utrwalona w systemie razem z raportem przy zatwierdzeniu w scenariuszu Create Scouting Report.
Wyzwalacz	Scout, w trakcie wypełniania formularza raportu skautingowego, decyduje się wprowadzić kolejną ocenę szczegółową w wybranej kategorii.
Warunki początkowe	• Przypadek użycia Create Scouting Report jest aktywny - formularz nowego raportu jest otwarty w trybie edycji. • Sekcja ocen szczegółowych formularza jest dostępna dla Scouta.
Warunki końcowe - sukces	• W formularzu tworzonego raportu pojawia się nowa pozycja Detailed Rating z polami: type, rating: [1, 10], comment oraz weight: [0.0, 1.0]. • Pozycja zostaje dołączona do lokalnej kolekcji ocen formularza (bufor w pamięci) - jeszcze bez utrwalania w systemie. • Faktyczne utworzenie obiektu Detailed Rating w systemie nastąpi wyłącznie wraz z zatwierdzeniem nadrzędnego Scouting Report w scenariuszu Create Scouting Report (relacja kompozycji - Detailed Rating nie istnieje bez Scouting Report).
Warunki końcowe - porażka	• Lokalny stan formularza nie zmienia się - żadna nowa pozycja Detailed Rating nie zostaje dołączona do sekcji ocen szczegółowych raportu. • W systemie nie powstaje żaden obiekt Detailed Rating.

7.2.2 Powiązane przypadki użycia

- <<include>> from Create Scouting Report - przypadek użycia jest dołączany w trakcie tworzenia raportu skautingowego (jest jego obowiązkową częścią - raport musi zawierać 1..* ocen szczegółowych). Scenariusz może zostać wykonany wielokrotnie w obrębie jednego wywołania Create Scouting Report.

7.2.3 Scenariusz główny

Scout nie zatwierdza pojedynczej oceny - wprowadzone wartości pozostają w buforze formularza, a właściwy zapis ocen w systemie następuje dopiero przy zatwierdzeniu raportu (**Submit Report**) w scenariuszu **Create Scouting Report**. Walidacja zakresów odbywa się po stronie frontendu w trybie ciągłym (live), natomiast wiążąca walidacja systemowa wykonywana jest dopiero w momencie **Submit Report**.

1. Scout wybiera w sekcji ocen szczegółowych formularza akcję „+ Dodaj ocenę”.
2. System dodaje do sekcji ocen nowy pusty wiersz oceny szczegółowej z polami: **type**, **rating**, **comment**, **weight** i uruchamia ich walidację wizualną w trybie ciągłym.
3. Scout wprowadza wartość pola **type** (kategoria swobodna, np. „*offense*”, „*defense*”, „*athleticism*”, „*basketball IQ*”, „*character*”).
4. Scout wprowadza wartość pola **rating** z zakresu [1, 10]. Frontend na bieżąco sygnalizuje wizualnie poprawność wartości - pole pozostaje w stanie błędu, jeżeli wartość wykracza poza zakres (*wskaźnik wyjątku E1*).
5. Scout wprowadza treść pola **comment** z uzasadnieniem oceny.
6. Scout wprowadza wartość pola **weight** z zakresu [0.0, 1.0]. Frontend na bieżąco sygnalizuje wizualnie poprawność wartości - pole pozostaje w stanie błędu, jeżeli wartość wykracza poza zakres (*wskaźnik wyjątku E1*).
7. Wprowadzona pozycja pozostaje w sekcji ocen szczegółowych formularza jako element bufora tworzonego raportu - bez utrwalania w systemie.
8. Scout może powtórzyć kroki 1-7 dla kolejnych ocen szczegółowych w innych kategoriach (1..* pozycji w obrębie jednego raportu).
9. Przypadek użycia kończy się sukcesem - sterowanie wraca do scenariusza **Create Scouting Report**. Faktyczne utworzenie obiektów **Detailed Rating** nastąpi dopiero przy zatwierdzeniu raportu (**Submit Report**).

7.2.4 Scenariusze wyjątków

7.2.4.1 E1. Wartość oceny lub wagi poza dopuszczalnym zakresem (walidacja wizualna)

W kroku 4 lub 6 scenariusza głównego, jeżeli wprowadzona wartość nie spełnia warunku **rating**: [1, 10] lub **weight**: [0.0, 1.0]:

1. Frontend wyświetla błąd w czerwonej ramce z błędami: „Ocena musi mieścić się w zakresie 1-10. Waga musi mieścić się w zakresie 0.0-1.0.”
2. Scout może kontynuować edycję pozostałych pól, jednak nieprawidłowa wartość nie zostaje wyczyszczona automatycznie.
3. Wiążąca walidacja systemowa wykonywana jest w scenariuszu **Create Scouting Report** przy **Submit Report** - jeżeli pozycja nadal zawiera nieprawidłową wartość, raport zostaje odrzucony zgodnie z wyjątkiem E4 scenariusza **Create Scouting Report**.
4. Scenariusz pozostaje odpowiednio w kroku 4 lub 6 do czasu poprawienia wartości lub usunięcia pozycji oceny (*wyjątek E2*).

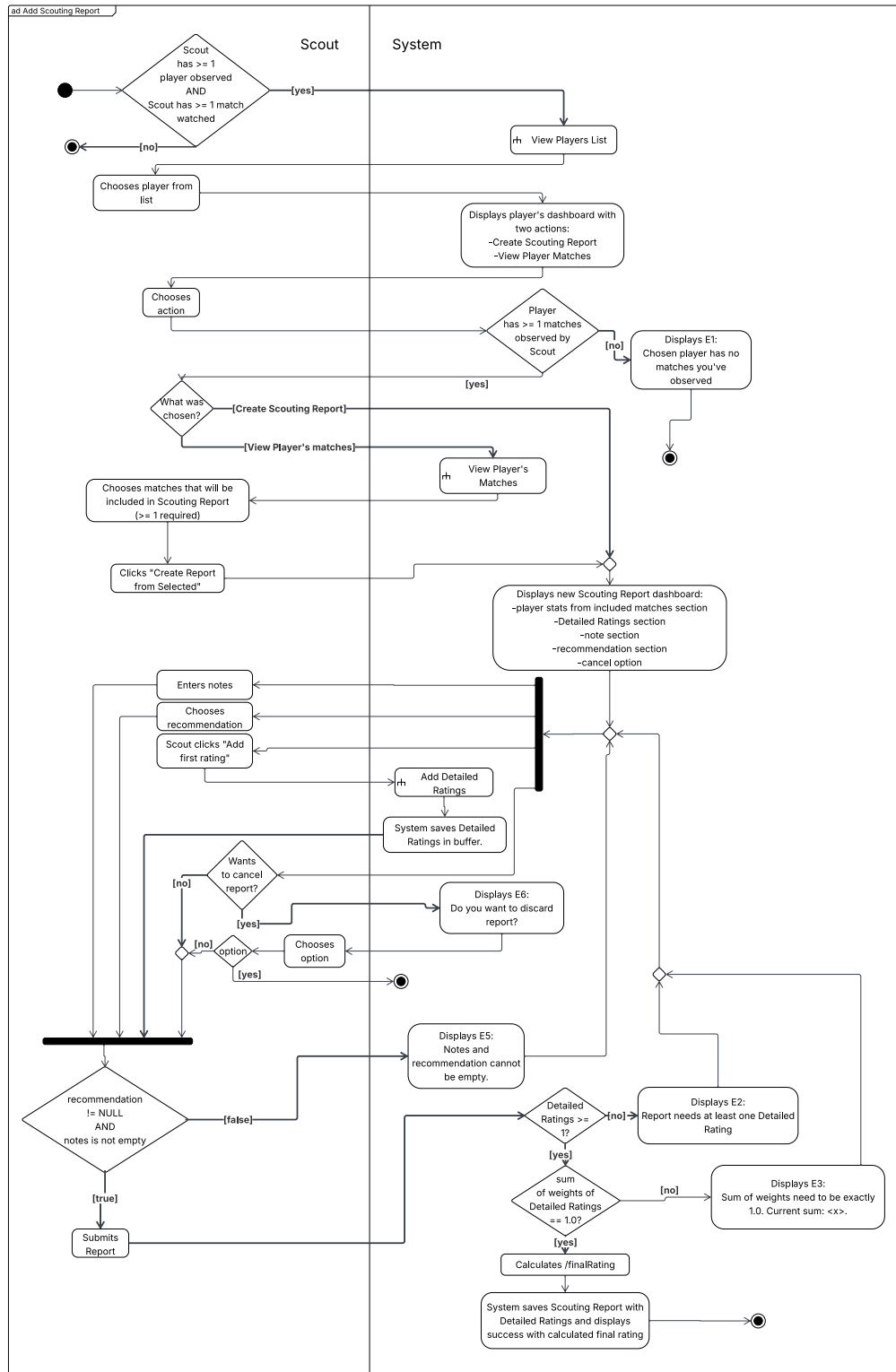
7.2.4.2 E2. Scout usuwa rozpoczętą ocenę

W dowolnym kroku po kroku 1 scenariusza głównego:

1. Scout wybiera w wierszu oceny akcję „*Usun*„.
2. System usuwa wiersz oceny z sekcji ocen szczegółowych formularza wraz z wprowadzonymi wartościami pól.
3. Lokalny stan formularza nie zawiera tej pozycji - pozostałe wprowadzone wcześniej oceny pozostają nienaruszone.
4. Przypadek użycia kończy się niepowodzeniem (warunki końcowe - porażka) dla tej konkretnej pozycji, sterowanie wraca do scenariusza **Create Scouting Report**.

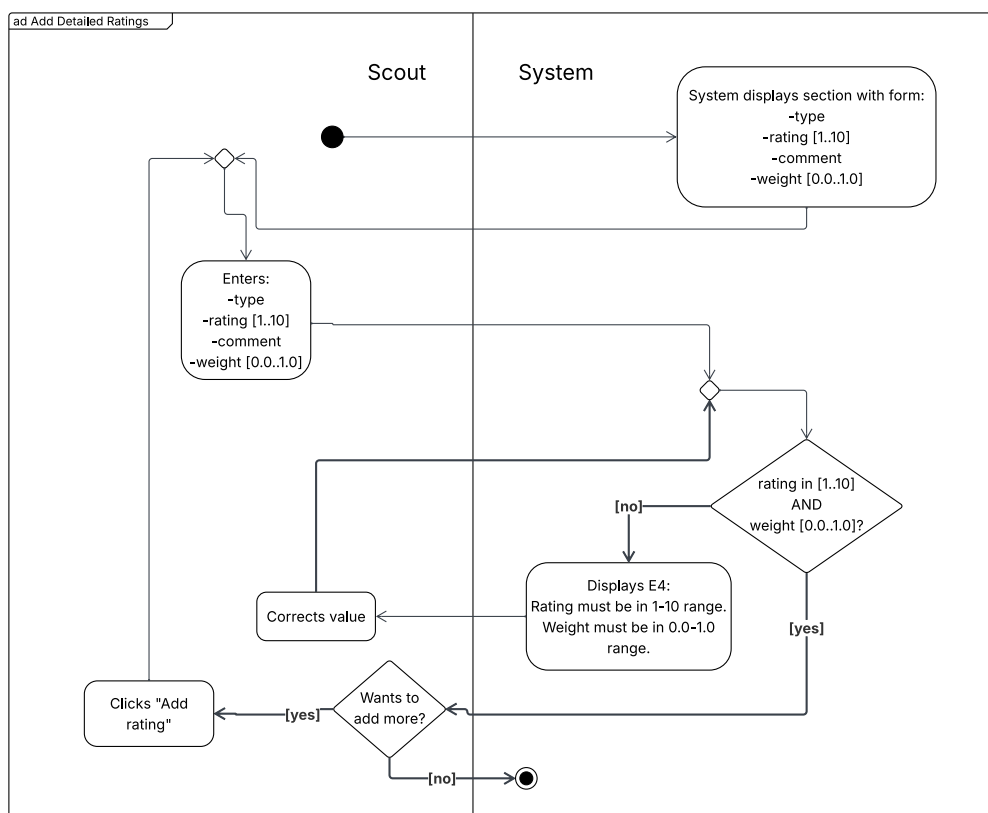
8 Diagramy aktywności

8.1 Create Scouting Report



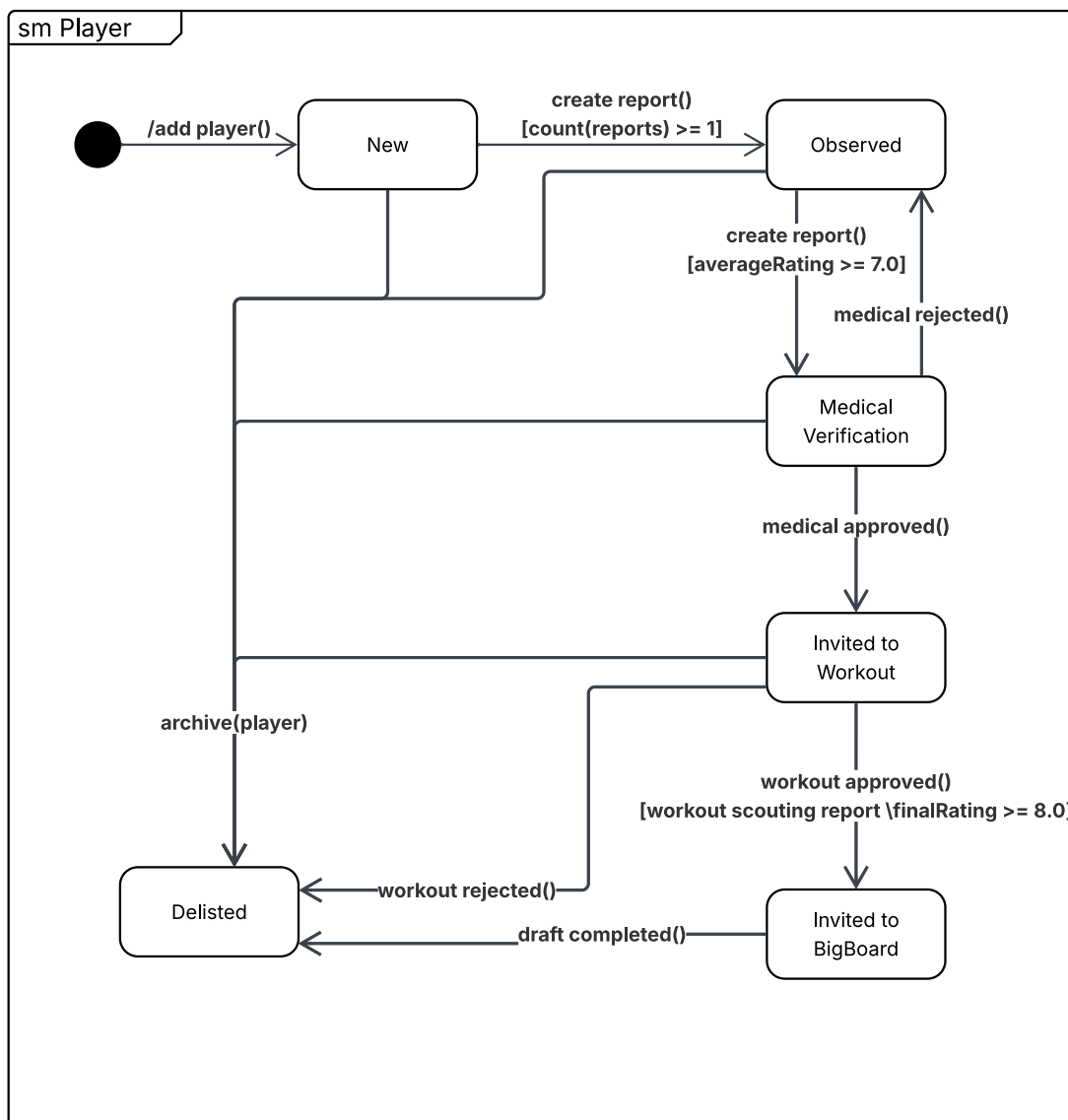
Rysunek 4: Diagram aktywności dla przypadku użycia „Create Scouting Report„

8.2 Add Detailed Ratings



Rysunek 5: Diagram aktywności dla przypadku użycia „Add Detailed Ratings,,

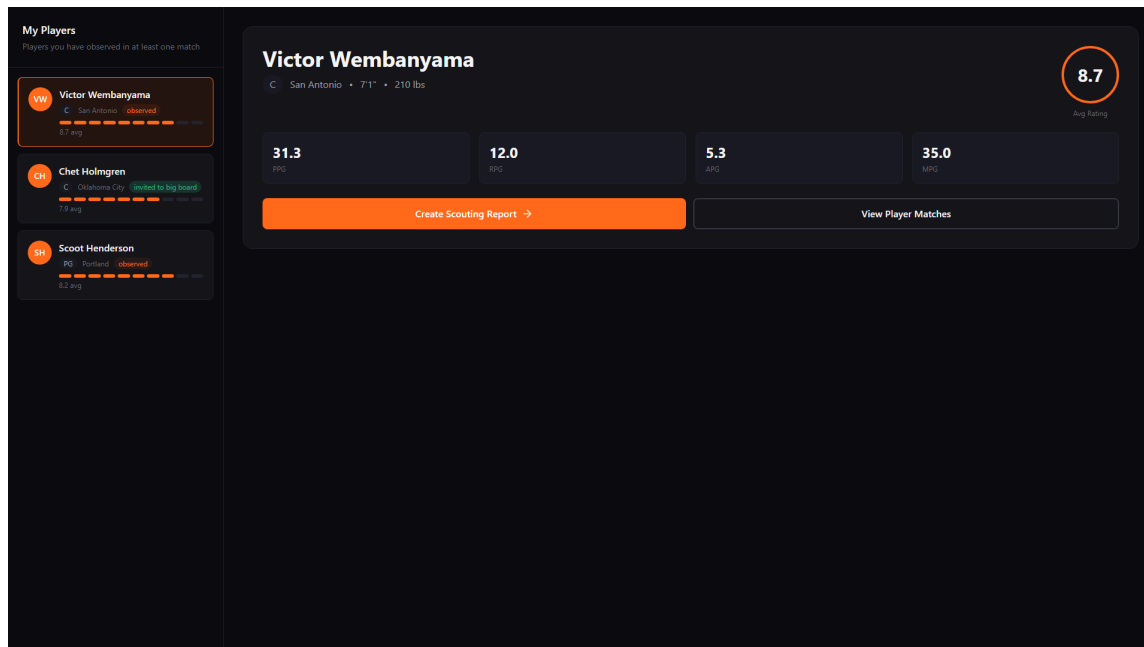
9 Diagram stanu dla klasy **Player**



Rysunek 6: Diagram stanu dla klasy **Player** (atrybut **player_status**)

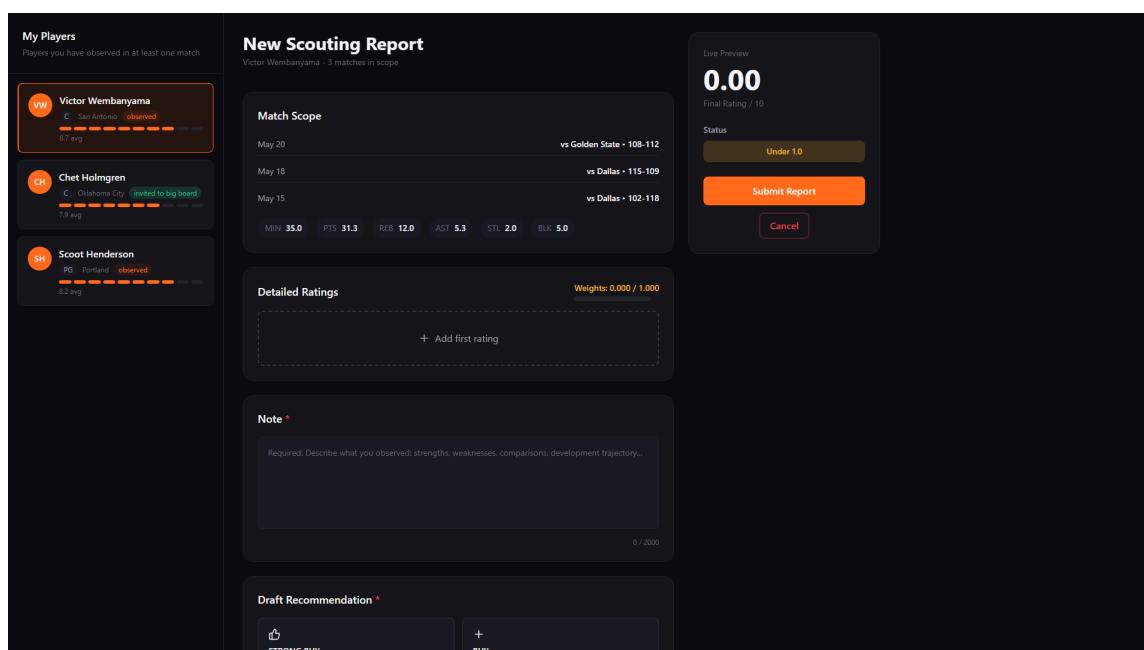
10 Projekt interfejsu użytkownika (GUI)

10.1 Ekran główny aplikacji po kliknięciu w zawodnika



Rysunek 7: Ekran główny aplikacji ScoutForce po kliknięciu w zawodnika z listy zawodników po lewej stronie.

10.2 Create Scouting Report



Rysunek 8: Ekran tworzenia raportu skautingowe zawodnika - widzimy statystyki zawodnika, sekcję z Detailed Ratings oraz Notes.

My Players

Players you have observed in at least one match

VW

Victor Wembanyama

San Antonio

observed

6.7 avg

CH

Chet Holmgren

Oklahoma City

invited to big board

7.9 avg

SH

Scot Henderson

Portland

observed

8.2 avg

New Scouting Report

Victor Wembanyama - 3 matches in scope

Type

defense

Rating (1-10)

5

Weight (0.0-1.0)

1.2

Comment *

Required: Add your observations...

+ Add rating

Auto-balance weights

Note *

Required: Describe what you observed: strengths, weaknesses, comparisons, development trajectory...

0 / 2000

Draft Recommendation *

👍

STRONG BUY

high-priority target. Aggressive pursuit.

+

BUY

Clear interest. Continue tracking.

○

NEUTRAL

Wait-and-see.

✖

PASS

Not a fit at this stage.

Live Preview

7.20

Final Rating / 10

Rating Breakdown

lebronism

3

× 0.40

defense

5

× 1.20

Status

Over 1.0

Submit Report

Cancel

Rysunek 9: Ekran tworzenia raportu skautingowe zawodnika część dalsza z Recommendation do wyboru

My Players

Players you have observed in at least one match

VW

Victor Wembanyama

San Antonio

observed

6.7 avg

CH

Chet Holmgren

Oklahoma City

invited to big board

7.9 avg

SH

Scot Henderson

Portland

observed

8.2 avg

New Scouting Report

Victor Wembanyama - 3 matches in scope

Type

defense

Rating (1-10)

5

Weight (0.0-1.0)

0.5

Comment *

amazing defense

+ Add rating

Auto-balance weights

Note *

wow, astonishing player

23 / 2000

Draft Recommendation *

👍

STRONG BUY

high-priority target. Aggressive pursuit.

+

BUY

Clear interest. Continue tracking.

○

NEUTRAL

Wait-and-see.

✖

PASS

Not a fit at this stage.

Live Preview

4.00

Final Rating / 10

Rating Breakdown

lebronism

3

× 0.50

defense

5

× 0.50

Recommendation

STRONG BUY

Status

✓ Valid

Submit Report

Cancel

Rysunek 10: Formularz wypełniony, spełniający wymogi do wysłania.

10.3 Stany walidacji przy zatwierdzaniu raportu

My Players
Players you have observed in at least one match

- VW Victor Wembanyama**
C - San Antonio - observed
8.7 avg
- CH Chet Holmgren**
C - Oklahoma City - **Added to big board**
7.9 avg
- SH Scoot Henderson**
PG - Portland - observed
8.2 avg

New Scouting Report
Victor Wembanyama - 3 matches in scope

Please fix the following errors:

- E3: Weights must sum to exactly 1.0. Current: 1.600
- E4: Weight #2 must be between 0.0-1.0
- E4: Comment for rating #2 is required
- E5: Note is required
- E5: Draft recommendation is required

Match Scope

May 20	vs Golden State • 108-112
May 18	vs Dallas • 115-109
May 15	vs Dallas • 102-118

MIN 35.0 PTS 31.3 REB 12.0 AST 5.3 STL 2.0 BLK 5.0

Detailed Ratings Weights: 1.600 / 1.000

Type: lebronism

Rating (1-10): 3 Weight (0.0-1.0): 0.4

Comment: lebron

Type: defense

Rating (1-10): 3 Weight (0.0-1.0): 0.5

Live Preview
Final Rating / 10: 7.20

Rating Breakdown
lebronism 3 × 0.40
defense 5 × 1.20

Status: Over 1.0

Submit Report

Cancel

Rysunek 11: Stany walidacji formularza (wyjątki E2-E5). Komunikaty błędów: brak ocen szczegółowych (E2), suma wag $\neq 1.0$ (E3), wartości poza zakresem (E4), brak wybranej rekomendacji oraz pusta notatka (E5). System blokuje zapis do momentu spełnienia wszystkich warunków.

10.4 Potwierdzenie anulowania raportu (E6)

My Players
Players you have observed in at least one match

- VW Victor Wembanyama**
C - San Antonio - observed
8.7 avg
- CH Chet Holmgren**
C - Oklahoma City - **Added to big board**
7.9 avg
- SH Scoot Henderson**
PG - Portland - observed
8.2 avg

New Scouting Report
Victor Wembanyama - 3 matches in scope

Please fix the following errors:

- E3: Weights must sum to exactly 1.0. Current: 1.600
- E4: Weight #2 must be between 0.0-1.0
- E4: Comment for rating #2 is required
- E5: Note is required
- E5: Draft recommendation is required

Match Scope

May 20	vs Golden State • 108-112
May 18	vs Dallas • 115-109
May 15	vs Dallas • 102-118

MIN 35.0 PTS 31.3 REB 12.0 AST 5.3 STL 2.0 BLK 5.0

Detailed Ratings Weights: 1.000 / 1.000

Type: lebronism

Rating (1-10): 3 Weight (0.0-1.0): 0.5

Comment: lebron

Type: defense

Rating (1-10): 3 Weight (0.0-1.0): 0.5

Live Preview
Final Rating / 10: 4.00

Rating Breakdown
lebronism 3 × 0.50
defense 5 × 0.50

Recommendation: STRONG BUY

Status: Valid

Submit Report

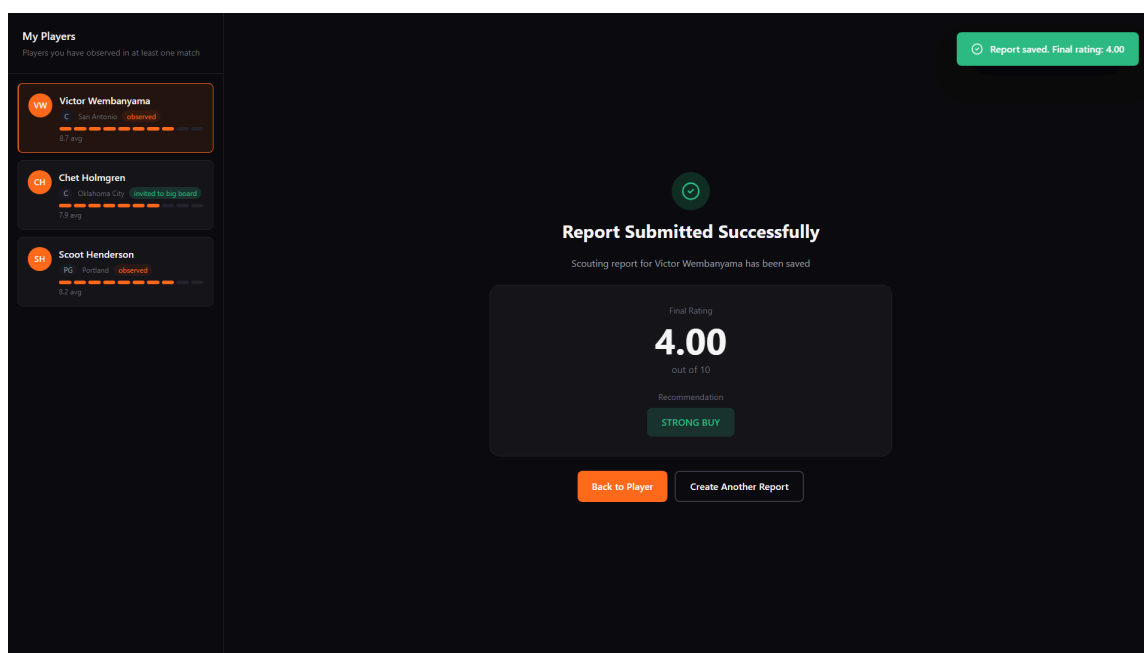
Cancel

Discard this report?
Your inputs will be lost. This cannot be undone.

Keep editing Discard report

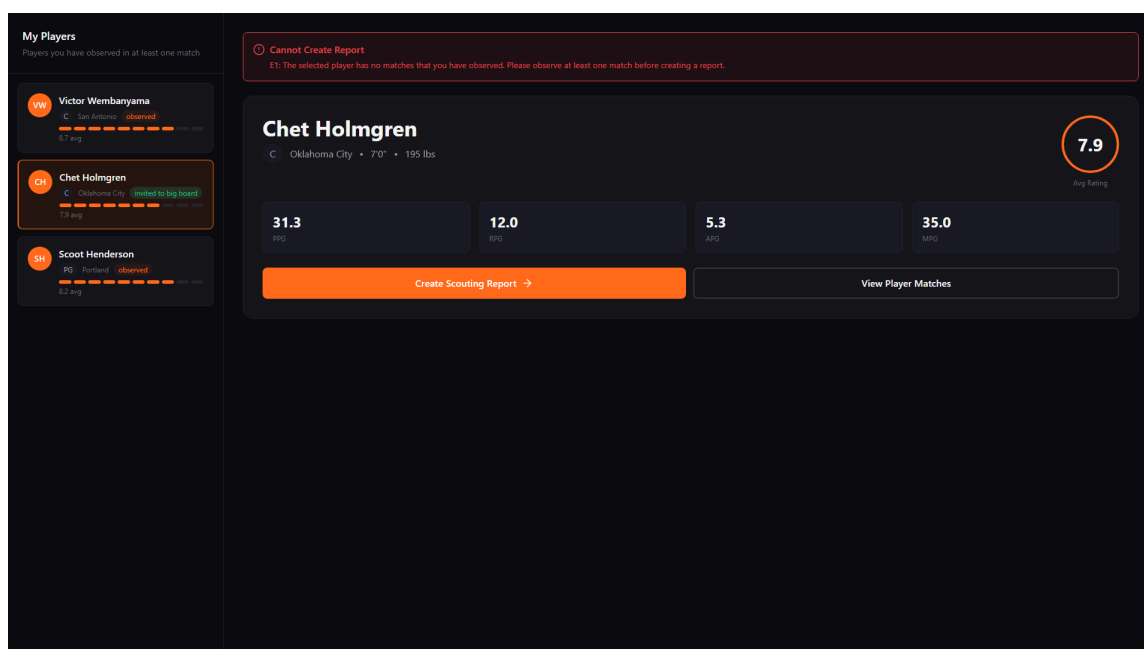
Rysunek 12: Dialog potwierdzenia anulowania (wyjątek E6). Komunikat: „Czy na pewno chcesz anulować raport? Wprowadzone dane zostaną utracone.” Skaut może potwierdzić anulowanie lub wrócić do edycji.

10.5 Potwierdzenie zapisu raportu - sukces



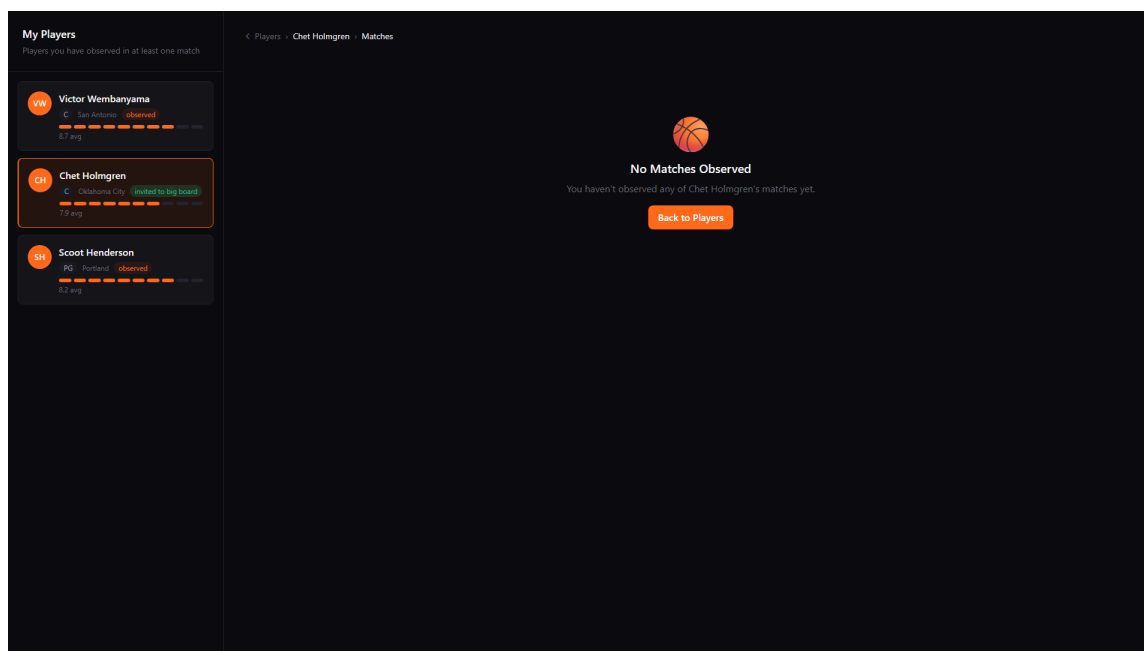
Rysunek 13: Ekran potwierdzenia zapisu raportu (kroki 14-15 scenariusza głównego). System wyświetla wyliczony `/finalRating` oraz podsumowanie zapisanego raportu. Przypadek użycia kończy się sukcesem.

10.6 Wyjątek E1 - brak meczów obserwowanych przez skauta (widok Create Report)



Rysunek 14: Stan pusty E1 - wybrany zawodnik nie ma żadnego meczu obserwowanego przez skauta. Komunikat: „Wybrany zawodnik nie ma żadnego meczu, który obserwowałeś. Aby utworzyć raport, najpierw zaobserwuj co najmniej jeden mecz tego zawodnika.” System wraca do panelu zawodnika.

10.7 Widok meczów zawodnika - brak meczów (scenariusz alternatywny A1 + E1)



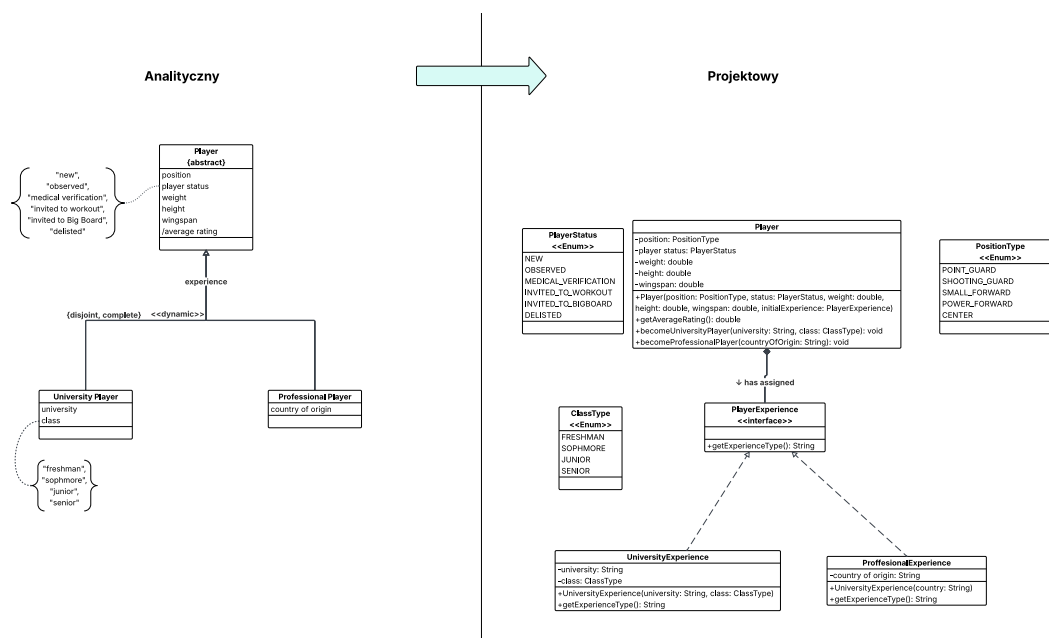
Rysunek 15: Widok „View Player Matches,” (scenariusz alternatywny A1, krok A1.2) w sytuacji, gdy lista meczów jest pusta - wyjątek E1. Skaut nie może utworzyć raportu z wybranych meczów, ponieważ żaden mecz nie spełnia warunków początkowych.

11 Omówienie decyzji projektowych i skutków analizy dynamicznej

11.1 Wykorzystane technologie

- **Backend:** Spring (Spring Boot + Spring Data JPA) z Hibernate jako dostawcą ORM.
- **Frontend:** React (SPA komunikujące się z backendem przez REST).
- **Baza danych:** H2 (wbudowana, prosta w konfiguracji do celów dydaktycznych; docelowo wymienna na dowolną relacyjną bazę zgodną z JDBC).
- **Modelowanie UML:** Lucidchart (diagramy przypadków użycia, klas, stanu, aktywności).
- **Projekt interfejsu (mockupy GUI):** Figma.

11.2 Klasa Player - dziedziczenie dynamiczne i ograniczenia atrybutów



Rysunek 16: Fragment projektowego diagramu klas - klasa Player wraz z interfejsem PlayerExperience oraz jego implementacjami

11.2.1 Dynamiczne dziedziczenie zawodnika (<<dynamic>>)

Z analizy wymagania nr 9 (rozdz. 2.3) wynika, że podział zawodników na **uniwersyteckich** (UniversityExperience) i **profesjonalnych** (ProfessionalExperience) jest **kompletny, rozłączny, ale dynamiczny** - zawodnik kończąc college przestaje być zawodnikiem uniwersyteckim i staje się profesjonalnym (lub odwrotnie, przy powrocie do NCAA). Klasyczne dziedziczenie statyczne w Javie tego nie obsłuży - obiekt nie zmienia swojej klasy w trakcie życia.

W projektowym diagramie klas zastosowano wzorec State / Strategy:

- Utworzono interfejs **PlayerExperience**, który jest **kompozycją** klasy **Player** - referencja do obiektu implementującego interfejs jest przekazywana w konstruktorze klasy **Player**, a klasa **Player** zarządza jej cyklem życia.
- Powstały dwie konkretne klasy implementujące interfejs: **UniversityExperience** (atrybuty: **university**, **class**) oraz **ProfessionalExperience** (atrybut: **countryOfOrigin**).
- Przepinanie aktualnej implementacji odbywa się przez metody klasy **Player**:
 - + **becomeUniversityPlayer(university: String, classType: ClassType): void**
 - + **becomeProfessionalPlayer(countryOfOrigin: String): void**

Dzięki temu jeden obiekt **Player** zachowuje swoją tożsamość (i powiązania z **Scouting Report**, **Match Stats** itd.), a zmienia jedynie strategię opisującą jego doświadczenie zawodnicze.

11.3 Ograniczenia atrybutów typu wyliczeniowego - enumy

W wyniku analizy dynamicznej wszystkie atrybuty o ograniczonej dziedzinie wartości zostały zamienione na **enumy Javy**. Zgodnie z konwencją nazewnictwa (rozdz. 1) stałe enumów są zapisywane w UPPERCASE.

PlayerStatus - status zawodnika w procesie skautingowym (atrybut **player_status** klasy **Player**):

- NEW
- OBSERVED
- MEDICAL_VERIFICATION
- INVITED_TO_WORKOUT
- INVITED_TO_BIGBOARD
- DELISTED

PositionType - pozycja zawodnika na boisku. Atrybut **position** (wymaganie 8) został w analizie dynamicznej dodatkowo ograniczony do skończonego zbioru wartości; każda pozycja ma przypisane wartości pomocnicze (skrót oraz numer pozycji 1-5), aby ułatwić skautom poruszanie się po systemie:

- POINT_GUARD("PG", 1)
- SHOOTING_GUARD("SG", 2)
- SMALL_FORWARD("SF", 3)
- POWER_FORWARD("PF", 4)
- CENTER("C", 5)

ClassType - rocznik zawodnika uniwersyteckiego. Atrybut **class** klasy **UniversityExperience** (wymaganie 10) został zamieniony na enum:

- FRESHMAN
- SOPHOMORE
- JUNIOR
- SENIOR

RecommendationType - rekomendacja draftowa raportu skautingowego. Atrybut **recommendation** klasy **Scouting Report** (wymaganie 16):

- STRONG_BUY
- BUY
- NEUTRAL
- PASS

DelegationStatus - status delegacji. Atrybut `status` klasy `Delegation` (wymaganie 20):

- PLANNED
- APPROVED
- IN_PROGRESS
- FINISHED
- CANCELLED

11.3.1 Sposób przechowywania enumów w bazie

W mapowaniu Hibernate wszystkie powyższe enumy są oznaczane adnotacją `@Enumerated(EnumType.STRING)` - w bazie H2 są przechowywane jako napis odpowiadający nazwie stałej (np. "INVITED_TO_BIG_BOARD"), a nie jako liczba porządkowa. Dzięki temu dodanie nowej wartości w przyszłości nie powoduje przesunięcia indeksów i nie psuje danych historycznych.

11.4 Mapowanie asocjacji i agregacji w Hibernate

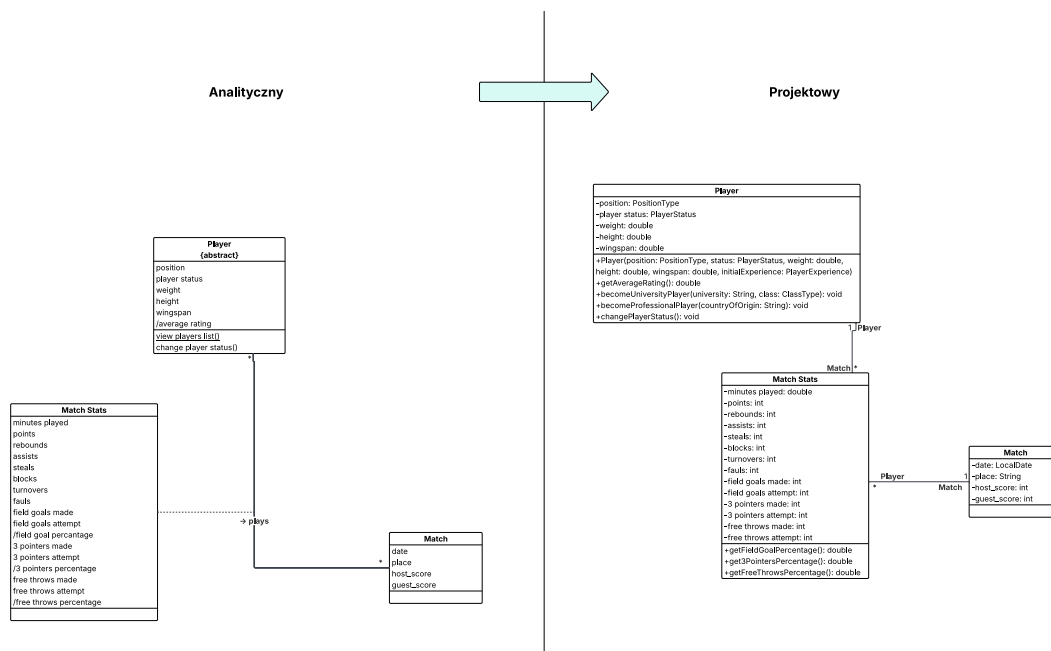
Wszystkie asocjacje z diagramu klas są realizowane za pomocą adnotacji JPA/Hibernate odpowiadających ich licznościom:

- **1 - *** → `@OneToMany` po stronie właściciela kolekcji oraz `@ManyToOne` po stronie elementów (np. `Delegation` → `Match`, `Scout` → `Scouting Report`).
- *** - *** → `@ManyToMany` (np. powiązanie meczów obserwowanych przez wielu skautów w ramach różnych delegacji).
- **Agregacja** jest implementowana w taki sam sposób jak zwykła asocjacja - z punktu widzenia Hibernate nie ma osobnej adnotacji dla agregacji; różnica między asocjacją a agregacją pozostaje informacją semantyczną na poziomie modelowania.

11.4.1 Ekstensja klas a persystencja (decyzja implementacyjna)

Na diagramie projektowym ekstensja klasy może być oznaczona metodami klasowymi. W implementacji ze Spring + Hibernate **nie będzie** utrzymywana jako `private static Set<T>` w encjach - stan trafi do bazy H2, a dostęp do zbiorów instancji oraz powiązań między obiektami będzie realizowany przez repozytoria i asocjacje JPA (patrz sekcja na końcu rozdziału).

11.5 Klasa Scouting Report - kompozycja, klasa pośrednicząca i atrybuty pochodne



Rysunek 17: Fragment projektowego diagramu klas - klasa Scouting Report wraz z Detailed Rating (kompozycja) oraz klasą pośredniczącą Match Stats

11.5.1 Klasa pośrednicząca Match Stats

W modelu analitycznym Match Stats reprezentowało asocjację wiele-do-wielu między Player a Match z dodatkowymi atrybutami (minutes_played, points, rebounds, assists, steals, blocks, turnovers, fouls, statystyki rzutowe). W projekcie taka konstrukcja nie istnieje wprost w Javie - została więc zastąpiona **klasą pośredniczącą** powiązaną z Player i Match dwiema asocjacjami **jeden-do-wielu**:

- Player 1 - * Match Stats,
- Match 1 - * Match Stats.

Każda instancja Match Stats reprezentuje pojedynczy występ konkretnego zawodnika w konkretnym meczu i nosi wszystkie atrybuty wcześniej leżące „na asocjacji”.

11.6 Kompozycja Scouting Report - Detailed Rating

Zgodnie z wymaganiem 17, ocena szczegółowa (Detailed Rating) nie istnieje bez raportu, który ją zawiera - usunięcie raportu skutkuje automatycznym usunięciem wszystkich ocen szczegółowych. Kompozycja zostanie zrealizowana w Hibernate przez:

- @OneToMany(mappedBy = "scoutingReport", cascade = CascadeType.ALL, orphanRemoval = true) po stronie Scouting Report,
- @ManyToOne(optional = false) po stronie Detailed Rating.

Kolekcja ocen szczegółowych w encji `ScoutingReport` (`List<DetailedRating>`) będzie persystowana przez Hibernate; dodawanie i usuwanie elementów odbywa się na tej kolekcji przed zapisem encji nadrzędnej.

11.6.1 Walidacja sumy wag - `validateDetailedRatings()`

Ograniczenie z wymagania 18 („suma wag wszystkich ocen szczegółowych w obrębie jednego raportu = dokładnie 1.0”) nie da się wyrazić wprost w UML jako licznosc ani w Hibernate jako ograniczenie kolumny. Zostanie zrealizowane jako **metoda obiektowa** klasy `Scouting Report`:

- `+ validateDetailedRatings(): void` - przegląda kolekcję `Detailed Rating`, sumuje wagi (`BigDecimal`) i rzuca wyjątek walidacyjny, jeżeli suma jest różna od 1.0 (z tolerancją dla błędów reprezentacji zmiennoprzecinkowej).

Metoda zostanie wywołana w serwisie aplikacyjnym przed `scoutingReportRepository.save(...)` (scenariusz `Create Scouting Report`, krok 13).

11.7 Atrybuty pochodne - metody `get{AtrybutPochodny}()`

Wszystkie atrybuty pochodne (oznaczone w diagramie prefiksem `/`) zostaną zaimplementowane jako **getterzy wyliczające wartość on-the-fly** na podstawie powiązanych obiektów, bez utrwalania w bazie (`@Transient` w Hibernate):

- `Player.getAverageRating(): BigDecimal` - średnia z `finalRating` wszystkich raportów zawodnika.
- `Scouting Report.getFinalRating(): BigDecimal` - średnia ważona ocen szczegółowych (`rating × weight`).
- `Delegation.getCost(): BigDecimal` - liczone według wzoru z wymagania 21.
- `Match Stats.getFieldGoalPercentage() / getThreePointPercentage() / getFreeThrowPercentage()` - procenty skuteczności (analogicznie dla `Shooting Analysis`).

11.8 Ograniczenia unikalności (`unique`)

Atrybuty oznaczone jako unikatowe (`email` w `Person`, `license_number` w `Scout`) zostaną zmapowane adnotacją `@Column(unique = true, nullable = false)` w Hibernate. Ograniczenie jest egzekwowane na poziomie bazy (constraint `UNIQUE`) - próba zapisu duplikatu kończy się wyjątkiem `DataIntegrityViolationException` przechwytywanym w warstwie serwisowej i tłumaczonym na czytelny komunikat dla użytkownika (NFR 6).

11.9 Implementacja atrybutów klasowych, dostępu do danych i metod obiektowych

Atrybuty klasowe (wymagania 22 i 23) - na encji `Delegation`:

- `private static final BigDecimal DAILY_LIVING_COST = 250`
- `private static final BigDecimal FLIGHT_COST = 1500`
- użycie w `getCost()` przy liczeniu kosztu delegacji

Dostęp do zbiorów obiektów (zamiast `static Set` w encji) - repozytoria Spring Data JPA, np.:

- `PlayerRepository` extends `JpaRepository<Player, Long>` - `findAll()`, zapytania po `player_status`, `position`,
- `ClubRepository` - `findByLeague(...)`,
- `ScoutingReportRepository`, `DelegationRepository` itd. według potrzeb przypadków użycia,
- zapis i usuwanie: `repository.save(entity)`, `repository.delete(entity)` (kaskady kompozycji obsłuży Hibernate).

Nawigacja po asocjacjach - pola kolekcji / referencji w encjach (`List`, `Set`) z adnotacjami JPA, np. `@OneToMany List<DetailedRating> detailedRatings` w `ScoutingReport`, `@OneToMany List<MatchStats> matchStats` w `Player`.

Metody obiektowe na encjach - m.in.:

- getterzy/setterzy pól oraz `@Enumerated` dla enumów,
- `Player.becomeUniversityPlayer(...)`,
`Player.becomeProfessionalPlayer(...)`, `Player.changeStatus(PlayerStatus)`,
- `ScoutingReport.validateDetailedRatings()` (wywołanie przed `save` w serwisie),
- `@Transient` getterzy pochodne: `getFinalRating()`, `getAverageRating()`, `getCost()`, procenty w `MatchStats` / `ShootingAnalysis`.